INSTITUTO TECNOLÓGICO SUPERIOR DE LA
REGIÓN DE LOS LLANOS

INGENIERÍA MECATRÓNICA

Ciberfisica

Actividad: U1A6. (20%) BITÁCORA TÉCNICA

Nombre: Angel Daniel Cisneros Perez

Docente: Osbaldo Aragón Banderas

Fecha: 2026-03-01

TECNM
TECNOLÓGICO NACIONAL DE
MÉXICO

Índice

Objetivo **1**
Descripción breve del pipeline (sensor→PC→dashboard) **1**
Enlace al repositorio y estructura **1**
Resumen de versiones (v0.1 y v0.2) **2**
v0.1 (baseline histórico – U1A4) 2
v0.2 (pipeline principal – U1A5) 2
Metadatos mínimos del pipeline (extracto de metadata_pipeline.md) **3**
Hardware 3
Muestreo 3
Telemetría 3
PC/Dashboard 3
Resumen de bitácora **4**
Decisiones técnicas 4
Incidentes relevantes (con solución) 4
Checklist de reproducibilidad **5**
Orden de ejecución recomendado 5
Verificaciones rápidas (pasa/falla) – evidencia real 5
Evidencias mínimas en el repositorio (rutas y capturas) **6**
Conclusiones **9**

Objetivo.

Documentar y versionar de forma reproducible el pipeline sensor→adquisición→telemetría→PC/dashboard→registro, registrando decisiones, versiones e incidentes, para garantizar trazabilidad y que cualquier persona pueda ejecutar el sistema y obtener el mismo resultado esperado.

Descripción breve del pipeline (sensor→PC→dashboard)

El sistema implementa un pipeline de adquisición de reflectancia para un seguidor de línea con 2 sensores TCRT5000 y un ESP32. Las lecturas analógicas se adquieren por ADC en el ESP32, se transmiten por Serial USB hacia una PC con Windows, y se visualizan/registran en un Notebook de Jupyter. El registro se guarda en CSV (muestras crudas) y se documentan parámetros de calibración en un archivo JSON. Adicionalmente, se incluye un script de verificación rápida para validar integridad temporal, muestreo y tamaño del dataset.

Enlace al repositorio y estructura

Repositorio (GitHub):
https://github.com/danilopert-ui/U1A6_Pipeline_Trazabilidad_CisnerosPerez

Estructura mínima (carpetas y archivos obligatorios):

- **`/code/`**
  - `/firmware/` (ESP32: `main.cpp`, `platformio.ini`)
  - `/pc_tools/` (Notebook + herramientas: `U1A5_Cisneros_Daniel.ipynb`, `verify_csv.py`)
  - `/node_red/` (histórico U1A4: `U1A4_node_red.json`)

- **`/data/`**
  - `/raw/` (`U1A6_sample.csv` con ≥300 muestras)
  - `/processed/` (`calibration_params.json` con min/max/umbral por canal)
- **`/docs/`**
  - `/captures/` (capturas de ejecución)
  - `/diagrams/` (diagrama del pipeline)
  - `verify_output.txt` (evidencia del verificador)
- **`README.md`**, **`CHANGELOG.md`**, **`metadata_pipeline.md`**, **`repro_checklist.md`**, **`bitacora.md`**, **`requirements.txt`**

Resumen de versiones (v0.1 y v0.2)

El repositorio incluye versionado real con commits claros y dos etiquetas (tags) para trazabilidad.

v0.1 (baseline histórico – U1A4)

- Pipeline de telemetría por Serial usando Node-RED (flow JSON).
- Enfoque en adquisición/visualización rápida en dashboard Node-RED.
- Tag: **`v0.1`**.

v0.2 (pipeline principal – U1A5)

- Pipeline principal con 2 sensores analógicos TCRT5000 y Notebook Jupyter como dashboard/calibración.
- Registro a CSV + archivo JSON de parámetros de calibración.
- Script de verificación rápida (**`verify_csv.py`**) y evidencia en **`docs/verify_output.txt`**.
- Tag: **`v0.2`**.

Metadatos mínimos del pipeline (extracto de metadata_pipeline.md)

Hardware

- MCU: ESP32 DevKit (30 pines).
- Sensores: 2× TCRT5000 (reflectivo IR), salida analógica **AO** hacia ADC.
- Alimentación del módulo: **3.3 V** (requisito: AO ≤ 3.3 V).
- Canales ADC: **`s1=GPIO34 (ADC1), s2=GPIO35 (ADC1).`**
- ADC: 12-bit (0–4095), atenuación **`ADC_11db.`**

Muestreo

- Periodo de muestreo (firmware): 100 ms.
- Frecuencia nominal: 10 Hz.
- Fuente de tiempo: **`t_ms = millis()`** (ms).

Telemetría

- Canal: Serial USB.
- Baudrate: 115200 bps.
- Formato por línea: **`t_ms,s1,s2\n.`**

PC/Dashboard

- SO: Windows.
- Herramienta: Jupyter Notebook (**`.ipynb`**) con controles (ipywidgets) y gráfica.
- Dependencias: ver **`requirements.txt.`**

Resumen de bitácora

Decisiones técnicas

1. **Canal de telemetría: Serial USB (115200 bps).**
   Se eligió por simplicidad de integración, baja latencia y facilidad de reproducir en Windows con un puerto COM configurable.
2. **Muestreo a 10 Hz (100 ms).**
   Se definió para equilibrar estabilidad, carga de CPU/serial y facilidad de visualización/logging en Notebook sin saturar la UI.
3. **Dashboard principal en Notebook (Jupyter) en v0.2.**
   Se eligió para integrar visualización en tiempo casi real + calibración + logging a CSV + verificación en una sola herramienta reproducible.

Incidentes relevantes (con solución)

1. **Dependencias faltantes en PC ("No module named pandas").**
   Causa: entorno de Python sin librerías instaladas.
   Mitigación: crear entorno virtual **`.venv`** e instalar con **`pip install -r requirements.txt`**. Evidencia: ejecución exitosa del verificador y salida guardada en **`docs/verify_output.txt`**.
2. **Saturación del ADC (valores frecuentes en 4095).**
   Observación: saturación alta en datos crudos, lo que degrada calidad de calibración si el sensor opera fuera de rango útil.
   Mitigación: registrar como limitación/alerta y mantener trazabilidad de parámetros (se conserva **`calibration_params.json`** y se reporta la saturación en la verificación).

Checklist de reproducibilidad

Ubicación: **`repro_checklist.md`**

Orden de ejecución recomendado

1. Preparación (Windows): identificar COM del ESP32 y preparar Python.
2. Instalar dependencias: **`pip install -r requirements.txt`** (preferible dentro de **`.venv`**).
3. Cargar firmware en ESP32 (**`code/firmware/`**).
4. Ejecutar Notebook (**`code/pc_tools/U1A5_Cisneros_Daniel.ipynb`**) y verificar gráfica/lecturas.
5. Registrar datos y guardar CSV (**`data/raw/U1A6_sample.csv`**).
6. Verificación rápida con script: **`verify_csv.py`** y evidencia en **`docs/verify_output.txt`.**

Verificaciones rápidas (pasa/falla) – evidencia real

Resultados del verificador (**`docs/verify_output.txt`**):

- Filas: **6363** (cumple ≥300).
- Orden temporal: **`t_ms`** monótono = **True**.
- **`dt_ms`**: min=100, median=100, max=100 (muestreo estable).
- **`fs_est`**: **10.000 Hz**.
- **`gaps (dt > 150 ms): 0`**.
- Saturación: **`s1`** 64.6%, **`s2`** 49.6% (registrado como incidencia/limitación).
- Estado: **PASS: verificación básica OK**.

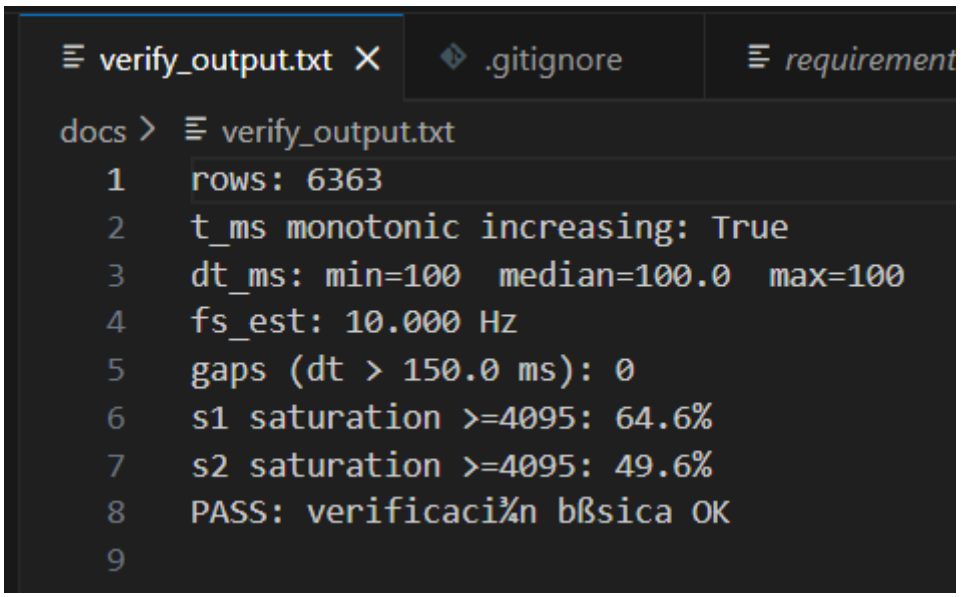

Fig 1. Verificación rápida del CSV. *Captura o evidencia del archivo* ***`docs/verify_output.txt`*** *con resultados de integridad temporal, fs estimada y conteo de filas.*

Evidencias mínimas en el repositorio (rutas y capturas)

- `docs/captures/salida_serial.png` (telemetría visible).
- `docs/captures/notebook_ejecucion.png` (dashboard notebook en ejecución).
- `docs/captures/parametros_calibracion.png` (parámetros/calibración).
- `docs/diagrams/diagrama.png` (diagrama pipeline).
- `data/raw/U1A6_sample.csv` (muestra ≥300; aquí 6363).
- `data/processed/calibration_params.json` (parámetros).
- `docs/verify_output.txt` (evidencia verificación).

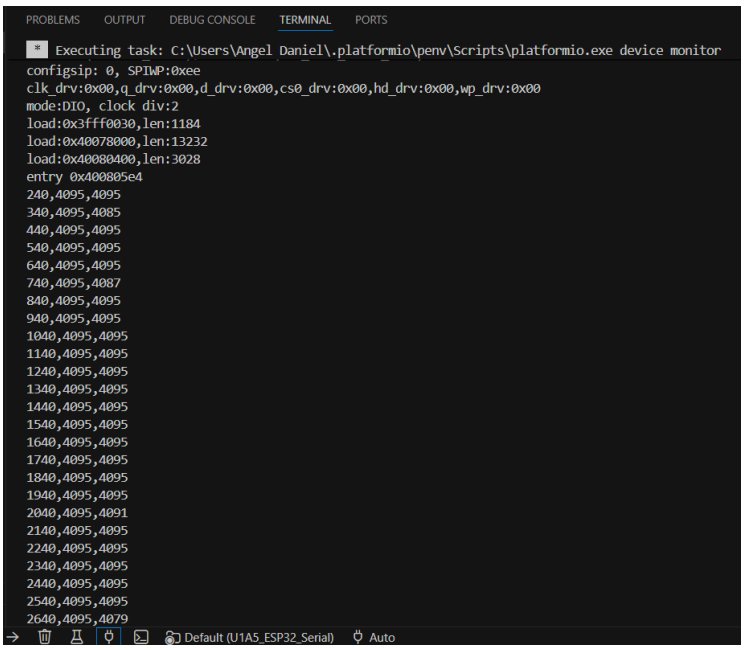

Fig 2. Salida Serial . *Evidencia de transmisión* `t_ms, s1, s2` *a 115200 bps.*

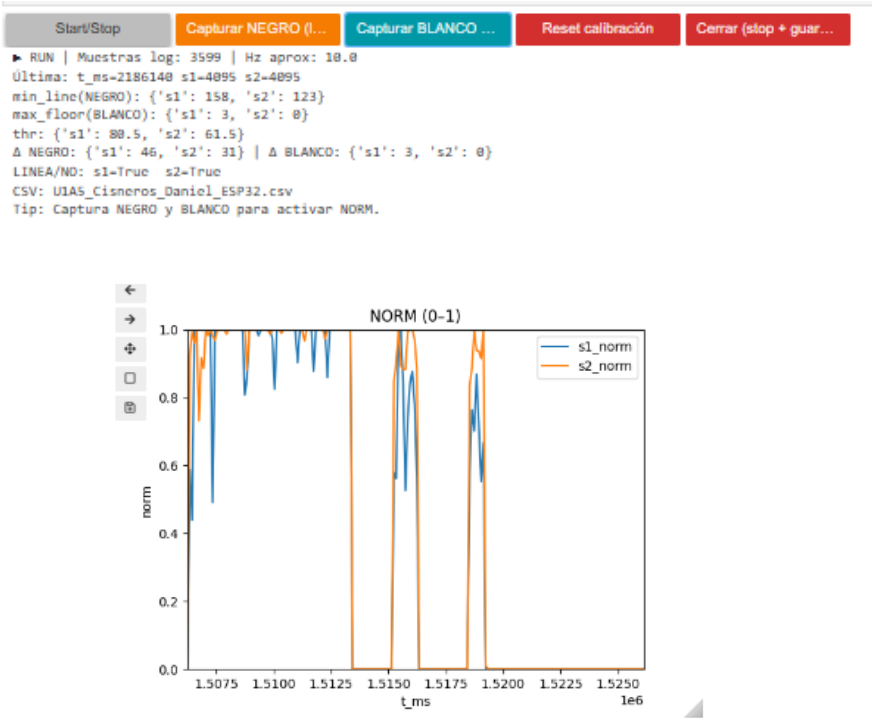

Fig 3. Notebook en ejecución. *Gráfica/controles y lectura en tiempo casi real desde COM.*

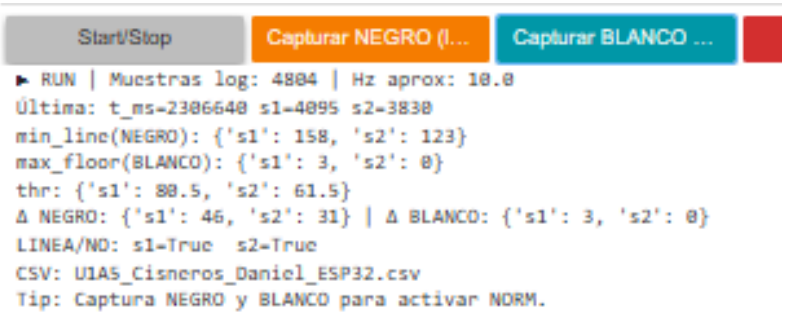

Fig 4. Parámetros de calibración. *Evidencia de umbrales/min/max (calibración) y estado de línea.*

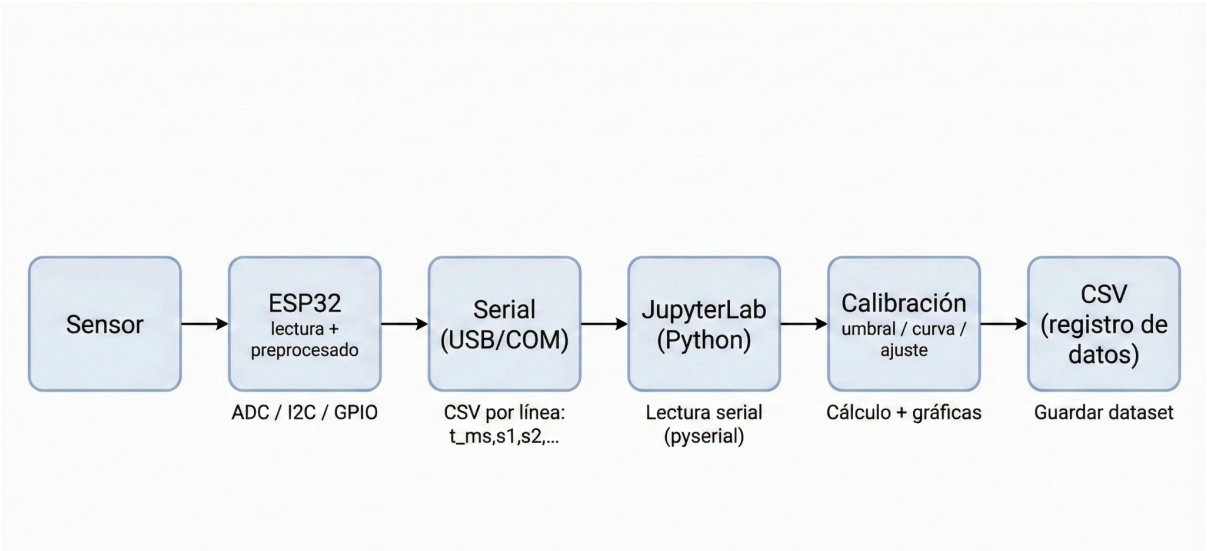

Fig 5. Diagrama del pipeline. *Flujo extremo a extremo sensor→ESP32→Serial→Notebook→CSV.*

Conclusiones

Se implementó un pipeline reproducible y trazable para adquisición de dos sensores TCRT5000 mediante un ESP32, transmitido por Serial USB y gestionado desde un Notebook en Windows. El repositorio incluye estructura requerida, bitácora, metadatos del pipeline, checklist de reproducibilidad, dataset de ejemplo ≥300 muestras (6363), parámetros de calibración y evidencia visual. El versionado está respaldado por commits y tags (`v0.1` y `v0.2`) que diferencian el baseline histórico (Node-RED) del pipeline principal (Notebook). La verificación automática confirma integridad temporal, muestreo estimado de 10 Hz y ausencia de huecos bajo el umbral definido; adicionalmente se documenta como incidencia la saturación de ADC para futuras mejoras mecánicas/ópticas del montaje.